

Report : M2-RDF-PROJECT

By Wouelebak Pierrette Daniela de Ombang & Martin Yeghiazaryan

1. Data Source

The data used in this report is sourced from the "Open Drug Knowledge Graph" dataset available on Kaggle. This is a relational dataset that includes information about drugs, treatments, conditions, and prices. The dataset can be accessed via the following link: [Open Drug Knowledge Graph](#)

2. Data Description

drug.csv: This file catalogs the names and properties of various medications. The column *drug_url* serves as the domain for each medication.

condition.csv: This file lists types of medical conditions along with their descriptions, which are linked to URLs. It has a relationship with the *source* table.

source.csv: This file provides the bibliographic references for the various medical conditions.

interactions.csv: This table acts as a relational mapping between the source of a drug and its target. In the graph model, *sourceDrug* and *targetDrug* will be defined as type, and the relationship will establish a link between the *sourceDrug* and *targetDrug* resources (*interactWith*).

product.csv: This file likely contains the names and characteristics of products derived from the drugs. It has relationships with the *source*, *manufacturer*, and *drug* tables.

manufacturer.csv: This file contains the names of the manufacturers responsible for producing the drugs.

price.csv: This file records the price of each product, depending on the respective store.

store.csv: This file contains the names of pharmaceutical establishments (stores).

treatment.csv: This file links the *source* table to the *condition* and *drug* tables, suggesting that a given source provides a treatment for a specific condition with the associated drug. The relationship "treatsBy" can be defined between the *condition* and *drug*.

3. Data Transformation (From csv to ttl)

To convert the raw CSV data into a well-labeled RDF TTL file, follow the steps below:

❖ Set Execution Permission for the Script

First, make sure that the `run_tarql.sh` script is executable. In your terminal, navigate to the project directory and run the following command:

```
chmod +x scripts/run_tarql.sh
```

❖ Run the `run_tarql.sh` Script

This script will execute all the necessary TARQL files and combine the results into a single output file `data/output/tarql_output.ttl`. To run the script, use the following command:

```
./scripts/run_tarql.sh
```

❖ **Run the Python Script for Label Transformation**

After executing the TARQL files, run the Python script `clean_tarql.py` to clean and label the output properly. This step is necessary to fix certain entity labels that may cause errors in the RDF file. To run the Python script, use the following command: `python3 scripts/clean_tarql.py`

❖ **Output**

After running both the shell and Python scripts, the final cleaned and labeled RDF output will be saved in the `data/output/rdf_ontology.ttl`.

4. Queries

• **Maximum price of products by store and manufacturer (query_aggreagation)**

This query retrieves the maximum price of each product sold in a given store, along with the associated manufacturer. The results are grouped by product, store, and manufacturer name.

• **Products, stores, and conditions treated (query_aggregation2)**

This query lists the top 5 products along with the stores they are sold in and the number of distinct conditions they treat. It organizes the results by grouping products and calculating the count of conditions treated, sorted in descending order.

• **Average product cost (query_aggregation3)**

This query calculates the average price of each product based on its cost in various stores. The results are grouped by product name and limited to the top 5 products.

• **Drugs and external information from Wikidata (query_external)**

This query retrieves drugs from Wikidata along with their English names and any available external information, such as URLs. It uses a federated query to access data directly from Wikidata.

• **Drugs and external information from Wikidata, duplicate (query_notexist)**

This is identical to Query 4, fetching drugs, their names, and external information from Wikidata.

• **Products, manufacturers, and optional reviews (query_optional)**

This query retrieves products, their names, and manufacturers, optionally including any reviews associated with the products. The results are limited to 5 entries.

• **Drug interaction network(query_paths)**

This query finds drugs that interact with one another through paths of length 2 or 3, listing both the source and target drug names. It highlights possible relationships between drugs within the specified limit.

For Query 1 and 3, we attempted to convert the `costs` data into numeric formats using `xsd:integer` and `xsd:decimal`, but these approaches were unsuccessful. Despite applying proper data type casting, the issue persisted, likely due to mismatches in the underlying data structure or format.

```

BIND(xsd:integer(?price) as ?price)#or
# Convert to decimal after replace
#BIND(REPLACE(?Price, "\\.", ",") AS ?price_with_comma)
#BIND(xsd:decimal(REPLACE(?Price, ",", ".")) AS ?price)

```

5. RDFS Schema

In the RDF schema provided, several constraints and relationships have been established to define the ontology of the data and its interactions:

- **Type Hierarchies and Subclass Relationships:**
 - `ex:TargetDrug` and `ex:SourceDrug` are defined as subclasses of `ex:Drug`, indicating that all instances of these types are also instances of the broader `ex:Drug` category. This hierarchy is useful for inferring broader relationships and categorizing drugs effectively.
- **Domain and Range Constraints:**
 - Properties such as `ex:interactsWith`, `ex:treatsBy`, and `ex:derivedFrom` have explicitly defined `rdfs:domain` and `rdfs:range`. For example:
 - `ex:interactsWith` has both its domain and range set to `ex:Drug`, meaning this property only applies to interactions between drugs.
 - `ex:treatsBy` links drugs (`ex:Drug`) to pathologies (`ex:Pathology`), indicating a treatment relationship.
 - `ex:derivedFrom` connects products (`ex:Product`) to the drugs they are based on, emphasizing the dependency between these entities.
 - Other properties, such as `ex:manufacturedBy`, `ex:indexedIn`, and `ex:sellsInStore`, specify the expected types for their range but not their domain, creating flexible but controlled relationships.
- **Minimal Constraints on Numerical Data:**
 - While `ex:costs` is associated with `ex:Product` via its domain, no additional constraints (e.g., data types like `xsd:decimal` or `xsd:integer`) were defined to enforce numeric or monetary formats. This lack of enforcement leaves room for potential inconsistencies in the data.

Limitations and Challenges

Despite these defined constraints, we were unable to identify or implement additional rules to further enrich our ontology or fact base. Some notable challenges include:

- **Lack of Specificity for Numerical Values:**
 - For properties like `ex:costs`, no specific range or format has been imposed (e.g., `xsd:decimal`), which limits data validation and precision in representing monetary values.
- **Absence of Cardinality Constraints:**
 - The schema does not define cardinality restrictions (e.g., limiting a product to being sold in one store or a drug being derived from a single source), leaving potential for data duplication or ambiguity.

- **No Explicit Temporal or Geographical Constraints:**
 - Relationships such as `ex:sellsInStore` or `ex:manufacturedBy` lack contextual information like time or location, which could provide greater detail about product availability or manufacturer operations.
- **Limited Use of Inference Rules:**
 - While subclass relationships and domain/range constraints enable some inferencing, more complex logic (e.g., OWL restrictions for disjoint classes or transitive properties) could add richer semantics to the data model.